TOOLBOX ARTICLE

MRFoundry: Knowledge-Guided Agent Loops for End-to-End MRI Research

Ke Wang

[1]Independent Researcher

Correspondence
Ke Wang. Email:
kewang0622@gmail.com

Abstract

Recent advances in AI agents have expanded their ability to write code, use software, and revise computational workflows, creating an opportunity to assist MRI research. Applying these capabilities requires connecting acquisition physics, data conventions, specialized tools, and scientific validation. We present MRFoundry, an open-source toolbox that makes this knowledge available to existing agents through seven complementary skills. The skills connect scientific references and established tools to concrete actions and output checks, while project-local memory preserves decisions and evidence for reuse. Researchers can adapt guidance locally and propose generalizable improvements through reviewed contributions. We demonstrate guidance use in a later audit of a researcher-supervised sequence prototype and during acquired-data reconstruction, showing how an agent combines tools, inspects outputs, and records unresolved questions. Alongside MRFoundry, we introduce MRI Agent Bench as an accompanying evaluation resource and use paired offline comparisons across models and MRI task domains. MRFoundry improved aggregate verified completion on the Core benchmark; evidence on harder tasks was less conclusive. MRFoundry provides a practical foundation for connecting MRI knowledge, computational tools, and researcher supervision in agent-assisted research.

KEYWORDS:
MRI, agentic workflows, research software, reproducibility

WORD COUNT: 3315

1 | INTRODUCTION

AI agents can write code, operate software, and use execution feedback to carry out computational work[1,2,3,4]. Their application to scientific workflows creates an opportunity to help researchers connect specialized tools, inspect intermediate results, and revise an implementation as a question develops[5]. For MRI, useful assistance must extend across acquisition design, simulation, reconstruction, analysis, and reporting. These stages depend on different software environments but share assumptions about the underlying experiment.

MRI decisions connect acquisition physics, hardware constraints, data conventions, and computational models[6]. A reconstruction can execute successfully while using incorrect axes or sampling conventions. A valid sequence file can still require physical and numerical checks. The practical difficulty is therefore not simply finding a software package: the agent must establish what the inputs represent, preserve their meaning between applications, and choose evidence that supports interpretation. Much of this knowledge is distributed across publications, documentation, and laboratory experience.

The opportunity is to make that knowledge actionable at the point of a decision. A researcher asking for a sequence prototype needs more than generated code: the workflow must connect requested timing to an exported sequence and then to a simulation that reads it correctly. Similarly, a reconstruction request requires a clear relationship between measured k-space, a sampling mask, an encoding model, and a comparison image. These connections motivate guidance organized around research tasks.

MRFoundry organizes this knowledge into seven complementary skills for existing agents[7]. Each skill connects scientific references and software instructions to actions and output checks. A project-local notebook preserves the evidence behind decisions, and selected lessons can improve shared guidance through human review. The researcher supplies the question and constraints; the agent consults guidance and performs the computational work; established applications provide the numerical methods. Agent4MR couples sequence generation to physics-aware validation, while BCER supplies a controller for long MRI workflows with explicit artifact binding[8,9]. MRFoundry supplies reusable guidance across MRI domains, with execution and recovery remaining the host's responsibility.

We describe the toolbox and its contribution workflow, then illustrate researcher-supervised sequence construction with a later skill-guided audit and acquired-data reconstruction with guidance consulted during the run. Alongside MRFoundry, we introduce MRI Agent Bench and compare matched agents with and without the toolbox. Benchmark specifications and public tasks are maintained separately[10,11]. The comparison tests the frozen guidance package; it does not isolate individual skills or measure benefits from later memory reuse.

2 | METHODS

2.1 | Knowledge-guided research workflows

Figure 1 shows the intended loop. The researcher supplies a question, inputs, and constraints. The agent identifies relevant skills, plans actions, executes tools, and checks whether the outputs address the question. Failed checks can prompt revision of an implementation or its assumptions. MRFoundry describes this process in instructions; the host and researcher control execution and decide when further work is needed.

2.1.1 | Skill organization and MRI knowledge

Each skill has an entry point that identifies when it applies and links to focused instructions, references, and, where useful, helper scripts. Short instructions guide the next decision, while linked material supplies the underlying concepts or software details. This organization lets the agent consult relevant topics without loading every reference for every request. A skill is a readable instruction package that the host agent interprets in context. It is not a separate model or a callable reconstruction algorithm. The same entry point can therefore guide tasks with different data and constraints, while the specific commands and checks follow from the current request.

Table 1 groups the seven skills into two general and five specialist skills. The MRI navigator connects concepts, formats, software, and references across the field. The research-workflow skill connects study design, experiments, evaluation, and writing[7]. Specialist skills cover hardware, pulse-sequence design, conventional reconstruction, deep-learning reconstruction, and diffusion MRI. They address decisions such as choosing an encoding model, checking gradient conventions, or establishing reconstruction baselines. Skills can be combined when a task crosses domains: sequence design may require hardware limits and reconstruction conventions, while a learned reconstruction study requires conventional comparisons and a defensible evaluation design.

The reference index connects these instructions to annotated papers, textbooks, courses, and software documentation[31]. References explain why a method is appropriate; software instructions explain how to use a particular implementation. The listed coverage describes the toolbox's scope rather than demonstrated competence in every topic. For example, a reconstruction task can begin with the specialist skill, follow a format reference to establish array conventions, and consult

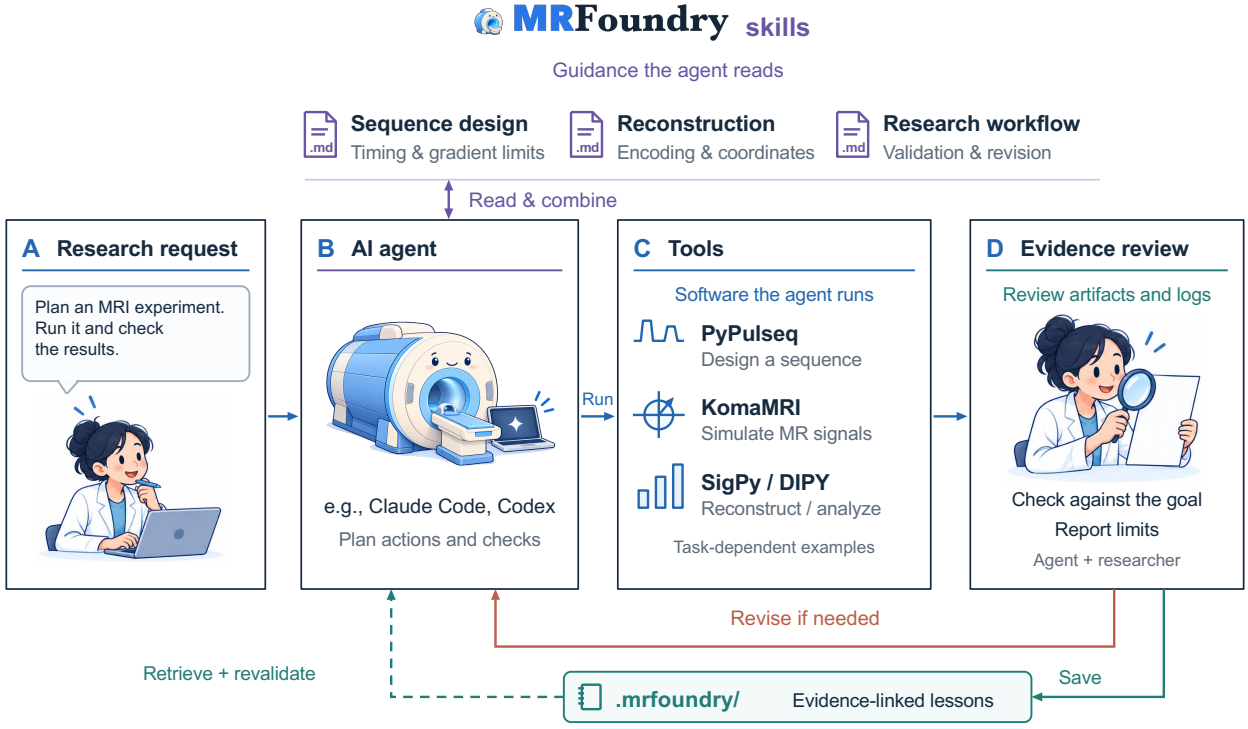


FIGURE 1 MRFoundry's guidance-based research loop[7]. A: the researcher specifies a goal. B: an AI agent consults relevant skills and plans actions and checks; Claude Code and Codex are illustrative hosts[12,13]. C: separately installed applications perform task-dependent computations, illustrated by PyPulseq, KomaMRI, SigPy and DIPY[14,15,16,17]. D: artifacts are reviewed with the researcher against the goal; insufficient evidence can prompt revision. The project-memory guide prescribes evidence-linked records, selective retrieval and revalidation[18]. Arrows describe recommended relationships rather than an enforced execution policy. Generated characters identify roles and are not scientific data. MRFoundry is compared with no skills in Table 2; the effect of project memory on later tasks was not measured.

TABLE 1 MRI-domain skills and reference entry points

MRI skill	Decisions supported	Reference entry points
MRI navigator `mri-research`	Physics, data conventions and cross-domain navigation	MRI and sequence foundations[6]
Research workflow `mri-research-workflow`	Study design, evidence review and reporting	Reproducible neuroimaging; AI reporting[19,20]
Hardware `mri-hardware`	Receive weighting and acquisition constraints	Receive-array foundations[21]
Sequence design `pulse-sequence-design`	Events, timing, trajectories and simulation	Pulseq, PyPulseq and KomaMRI[22,14,15]
Reconstruction `mri-reconstruction`	Encoding, calibration and regularization	SENSE, ESPIRiT and sparse MRI[23,24,25]; SigPy and BART[16,26]
Deep-learning reconstruction `deep-learning-recon`	Training, splits, baselines and generalization	fastMRI and end-to-end variational networks[27,28]
Diffusion MRI `diffusion-mri`	Gradients, model assumptions and fit interpretation	Diffusion tensor, DIPY and WLS[29,17,30]

The public source index links each stable skill name to its annotated references[31]. These are representative entry points. Skills supply guidance; separately installed applications compute. Topic coverage is not an evaluation result.

workflow guidance to define a baseline and endpoint. This use of linked references makes the relationship between an instruction and its scientific basis visible to both the agent and researcher.

2.1.2 | Agent workflow and tool use

Hosts such as Claude Code and Codex expose the skills to the agent[12,13]. The command `npx skills add KeWang0622/MRFoundry` installs the guidance; `--skill mri-reconstruction` selects one skill[32,7]. Scientific applications and restricted datasets retain their installation, access, and license requirements. MRFoundry does not replace those applications or implement their numerical solvers.

Before execution, guidance asks the agent to inspect the input data and working environment, resolve missing information, install dependencies in isolation, and check a small upstream example[33]. When connecting applications, the agent must preserve units, axes, sample order, and model assumptions. For example, reconstruction requires distinguishing the stored grid from acquired support before constructing a mask. After execution, the agent inspects artifacts and quantitative checks against the research request. A successful command is one intermediate observation; a usable result also needs a documented input interpretation, comparison reference, and explanation of remaining uncertainty.

2.2 | Research memory and shared guidance

Figure 2 distinguishes reusable public skills from project-local evidence. The `.mrfoundry/` notebook contains an index, working preferences, environment information, evidence-linked lessons, and individual run records[18]. Run records retain requests, input identifiers, versions, parameters, commands, outputs, and unsuccessful attempts. Keeping these details with the project allows another session to recover the reasoning behind a decision rather than only its final code.

A reusable lesson states the observation, supporting evidence, scope, and conditions for revalidation. Before later work, the agent reads the index, retrieves relevant lessons, and checks whether they apply to the current data and environment. It may retain, narrow, or reject a lesson. Conflicting evidence supersedes the earlier interpretation without erasing the original run. User preferences, such as a preferred reporting format, remain separate from scientific findings. This distinction prevents a local working habit from becoming an unsupported methodological rule.

This proposed recursive self-improvement cycle updates written guidance, not model weights. The initializer can add retrieval instructions to project `AGENTS.md` or `CLAUDE.md` files; the host controls their loading and execution[18]. Recording a retrospective is only the first step: improvement would require applying a lesson in subsequent work and checking the resulting outcome. We demonstrate evidence-linked recording, but did not evaluate later retrieval or its effect on task performance. The distinction matters because a recorded command can depend on a software version or source dataset. Reusing its conclusion without checking those conditions could repeat an error. The notebook therefore carries both reusable guidance and the information needed to decide when it should no longer be applied.

2.3 | Local adaptation and public contribution

Researchers can retain a lesson locally, adapt guidance to group conventions, or propose a generalizable change to the public toolbox. Project notebooks are excluded from Git by default. A public contribution selects the relevant instruction, removes private context, and supplies a scientific source or tested example[18,34]. The notebook itself is not automatically published.

Contributors edit the appropriate skill or reference on a branch and open a pull request (PR). Repository checks validate structure and links, while a maintainer reviews the content before merging[34]. A group can review the resulting update before adopting it. This separates an observation from one project from an instruction intended for broader use and gives corrections a traceable source. For instance, a source-specific array observation can motivate a general instruction to verify the release format, while its particular dimensions remain in the local record. Automated checks validate repository structure and look for broken links; scientific relevance and the scope of a proposed rule still require review. The contribution mechanism is implemented, but its effect on scientific quality or performance was not evaluated.

2.4 | Demonstration design

Two researcher-supervised Codex examples connect agent actions to scientific checks (Figures 3-4). Saved scripts, reports, and project records document tool use and revisions. Sequence skill retrieval occurred during a later audit, not original construction; reconstruction guidance reads occurred during the run. Neither example

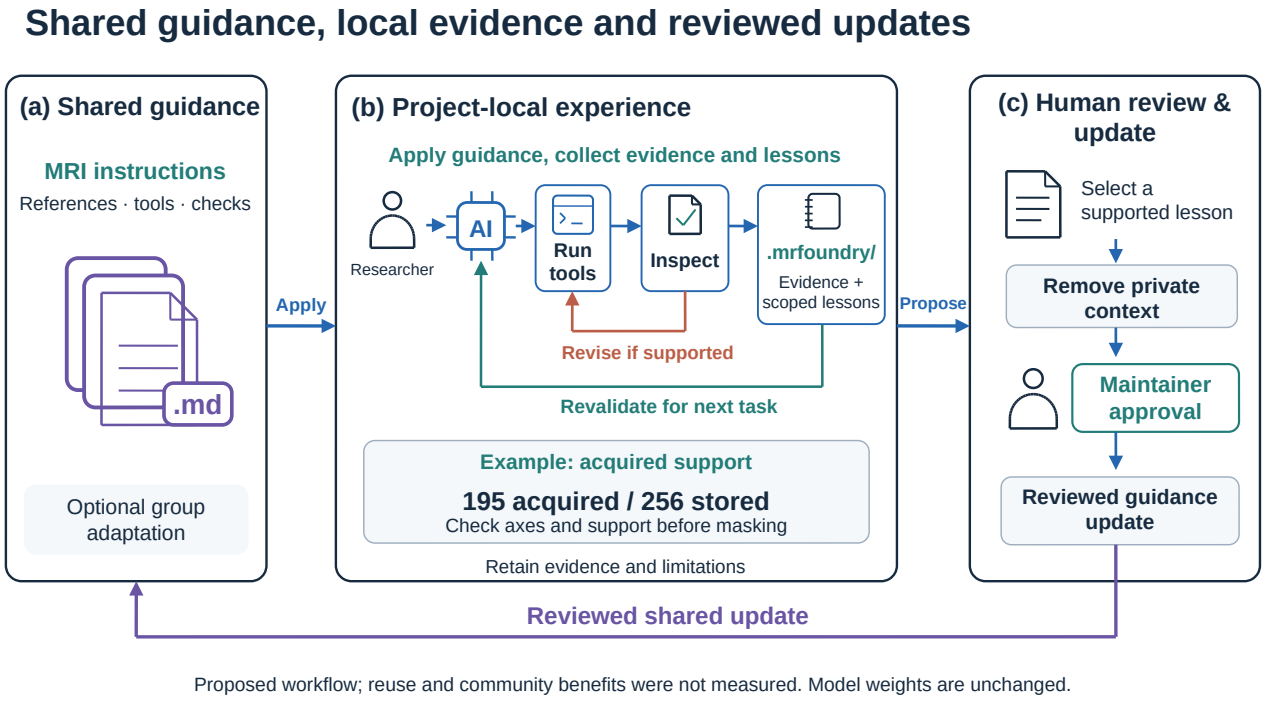


FIGURE 2 Shared guidance, local experience and reviewed improvement. (a) Public MRFoundry skills provide reusable MRI instructions and source links; a group can adapt these instructions to its methods and environment. (b) Project-local records retain preferences, evidence and scoped lessons. The researcher and agent execute tasks, inspect outputs and record evidence; current work can be revised when supported, while retrieved lessons must be revalidated for a new task. The acquired-support example distinguishes 195 acquired from 256 stored phase-encoding columns in the inspected source; the value is not a general acquisition rule. (c) Researchers select supported changes, remove private context and supply evidence for maintainer review before shared guidance is updated. Arrows describe the proposed guidance workflow rather than an enforced execution policy. This improvement loop was not evaluated in the fixed-release Core/Hard comparison; model weights are unchanged.

had a matched unguided baseline. Supporting Information retains numerical details, provenance, implementation history, and a separate diffusion example. NumPy, SciPy, and Matplotlib supported computation and display[35,36,37].

2.4.1 | Designing and checking a sequence

The task was to design, export, and inspect a golden-angle radial gradient-echo (GRE) prototype. PyPulseq 1.5.0.post1 produced a Pulseq file with a 220-mm field of view, 5-mm slice, 10-degree nominal flip, TE/TR 5/15 ms, and 128 spokes of 64 samples[14,22]. Successive spokes rotated by 111.246 degrees[38]. Configured per-axis gradient and slew limits were 32 mT/m and 130 T/m/s.

The agent constructed RF, gradient, and ADC events, checked timing and gradients, and compared trajectories before and after export. At the researcher's request, it replaced an initial custom simulator with KomaMRI, inspected the installed API, and ran the official EPI example before simulating the exported GRE on a 13,497-spin brain phantom[15]. Halving the RF integration step tested temporal discretization on the same phantom. An adapter preserved coordinates and phase when passing samples and trajectory to SigPy[16]. These steps examined the connections between sequence construction, exported representation, and simulated signal rather than relying on one final image. Reopening the exported file checked what another application would receive. Running an upstream simulator example checked the installed environment before introducing the study sequence. The RF-step comparison then examined whether one simulation setting materially changed the retained signal. Each check addressed a different possible source of implementation error.

2.4.2 | Comparing reconstructions

The task was to reconstruct an acquired four-channel, 0.3-T T2-weighted M4Raw brain slice at two retrospective sampling settings[39,40]. The agent consulted guidance, inspected the HDF5 array with h5py[41], and used the release contract to resolve axes and acquired

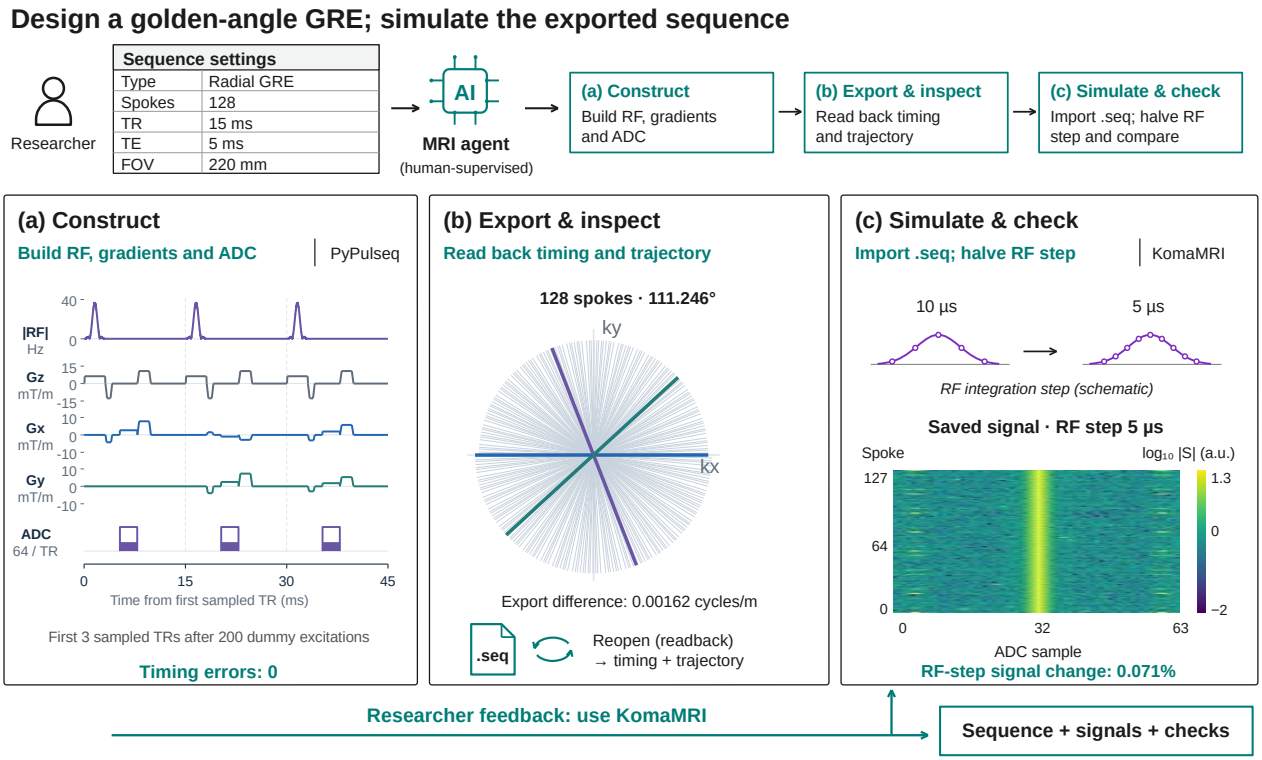


FIGURE 3 Researcher-supervised sequence construction followed by a skill-guided audit. The setup table combines agreed settings and the recorded 128-spoke implementation. (a) PyPulseq constructs RF, gradient and ADC events[14,22]. Native waveforms show the first three sampled repetitions after 200 dummy excitations; RF magnitude omits phase. Each repetition contains 64 ADC samples. Readback found no timing errors. (b) Exported spokes rotate by 111.246 degrees[38]; the maximum exported/prewrite coordinate difference is 0.00162 cycles/m. (c) At the researcher's request, the agent replaced an initial custom simulator with KomaMRI, inspected its installed API, ran its official EPI example and simulated the exported GRE[15]. The RF-step curves are illustrative sampling schematics. The heatmap displays log10 magnitude of the saved 5-μs-step signal, with zero-based spoke/ADC indices and the stated display range. Halving the RF step from 10 to 5 μs changed the complete complex signal by 0.071% (relative L2 change; exact value and definition in Supporting Information S2.1). Action labels summarize records rather than a verbatim transcript. Construction and simulation preceded the documented skill retrieval at audit time. These checks do not establish spatial convergence, image fidelity, SAR/PNS compliance or scanner readiness.

support[42]. It first ran SENSE and inspected image discrepancies and measurement residuals[23]. After checking calibration, it retained the ESPIRiT maps and tried L1-wavelet reconstruction with weight 0.01 at both masks[16,43,24,25,44].

The comparison preserved initial SENSE outputs and held sensitivity maps and reconstruction settings fixed between masks. Magnitude discrepancies used the noisy all-source root-sum-of-squares (RSS) image as reference. Sampled-line residuals assessed agreement with measurements used in the solve; withheld-line residuals used acquired measurements omitted from it. Additional iterations assessed numerical stability. These distinct checks made it possible to separate a stable optimization output from one that agreed with measured data. The same source-derived normalization and display window were used for the image comparison.

2.5 | MRI Agent Bench and comparison

MRI Agent Bench provides executable research tasks with specified inputs, artifacts, and automated checks (Figure 5). Public Core contains 24 task types across ten MRI domains and 88 data variants[10,11]. Coverage includes sequence design, reconstruction, quantitative imaging, analysis, and research planning and reporting. A type specifies a problem structure; an instance fixes data and parameters. The accompanying repository describes tasks and submission artifacts in detail. Separate author agents without access to the skills wrote specifications and private graders and generated data, parameters, and wording. Reference solutions and positive and negative controls were used to check instances before hashing and sealing them. This process checked the intended automated behavior; independent external MRI-expert review remains incomplete.

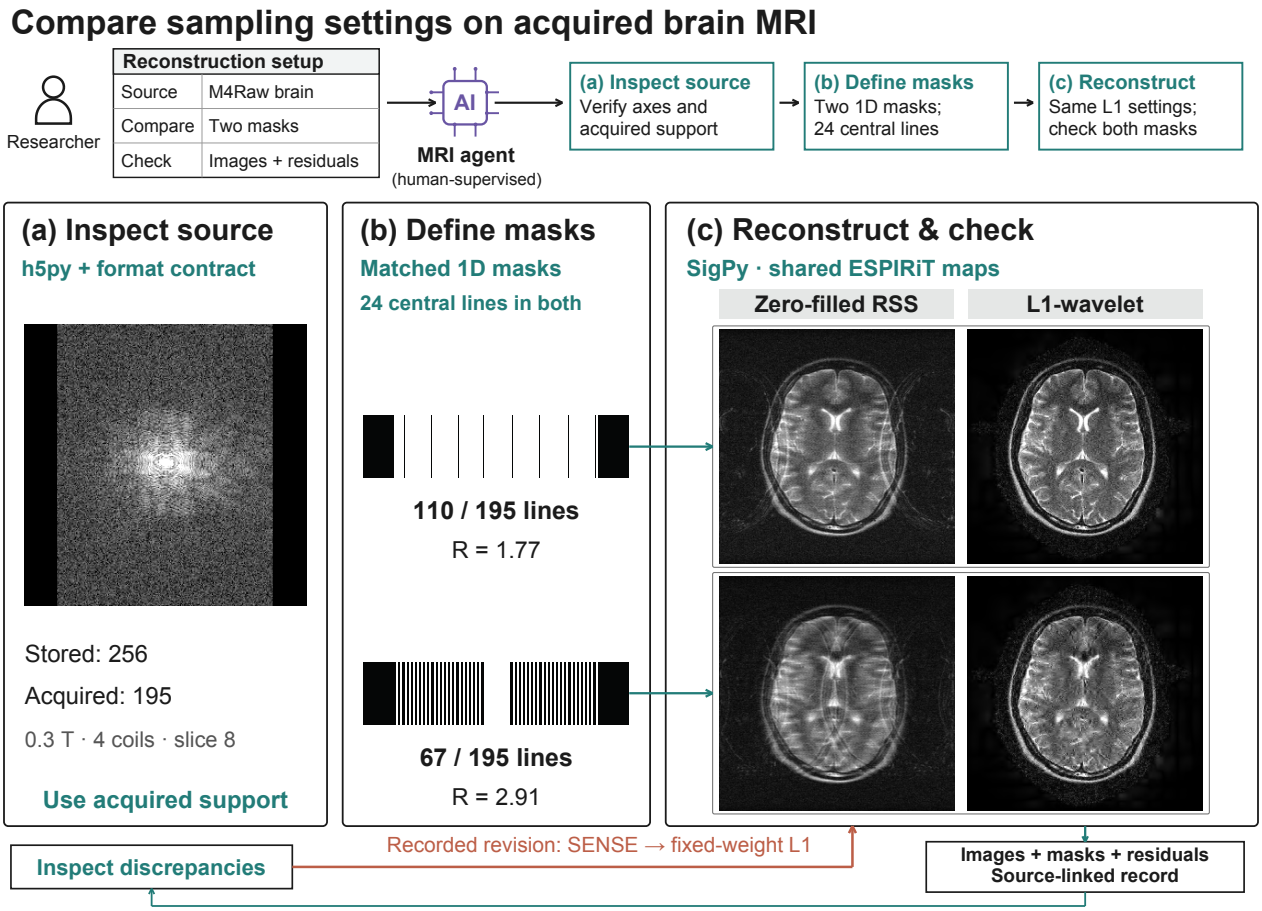


FIGURE 4 Recorded reconstruction workflow on one four-coil, 0.3-T M4Raw T2-weighted brain slice[39,40]. (a) The release contract identifies the final array axis as phase encoding; the source has 195 acquired lines in a 256-column grid. Grayscale k-space shows log10 root-sum-of-squared coil magnitudes, with fixed bounds −1.306 to −0.223 (1st-99.5th percentiles of nonzero acquired values; zero padding is black). (b) Exact retrospective masks retain 110 or 67 acquired lines, including 24 central lines, giving effective accelerations of 1.77 and 2.91. Each mask connects to a framed row containing both methods. (c) Saved zero-filled RSS and L1-wavelet images share source-RSS normalization and a window of 0-0.676 of the source RSS maximum, without per-image rescaling or inferred anatomical flips. Both L1 conditions use shared ESPIRiT maps, db4 wavelets, weight 0.01 and 200 iterations[16,24,25]. The agent revised initial SENSE to fixed-weight L1 after inspecting disagreement. Zero-filled RSS remained closer to the noisy reference, and substantial data residuals remained unresolved. Supporting Information retains the initial results and complete diagnostics. Action labels summarize records, not a transcript; the setup table combines agreed settings and recorded choices. The saved brain images retain their original pixels and common window. This supervised example is not a controlled agent-performance comparison.

Each pair ran concurrently through Codex in separate sandboxes with the same model, nominal reasoning setting, tools, inputs, and time limit[13]. Search was disabled; network access was limited to package indexes. The intervention comprised seven frozen skills plus their project-level discovery instruction; the agent chose whether to read them. The snapshot was fixed before evaluated Core instances were opened; subsequently authored Hard tasks were not used to revise it[45].

Ten models each attempted one sealed instance per Core type, giving 240 pairs rather than evaluating all 88 variants. For Hard, three models attempted two instances of all 15 types (90 pairs); four further models attempted one instance of 11 multi-stage types (44 pairs; Table 2). Four Hard types required broader system designs. Verified completion required every frozen automated check to pass. Checks examined submitted artifacts independently of the agent's report, including exported sequences, numerical outputs, code, and claims against supplied results. No language-model judge supplied completion labels.

The primary effect was the difference in paired completion proportions. A two-way task-type/model cluster bootstrap supplied 95% percentile intervals (4,000 resamples, seed 0), interpreted as sensitivity to the

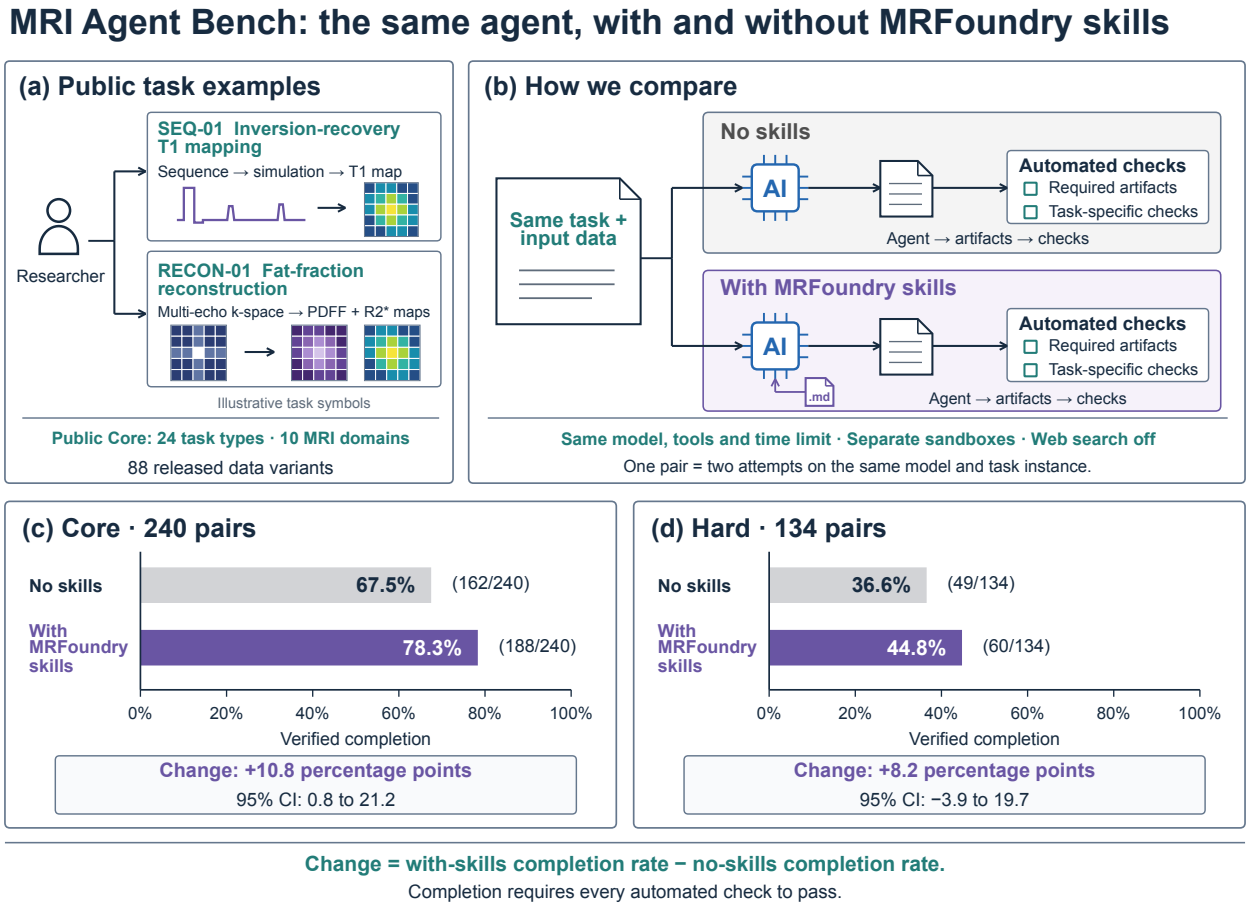


FIGURE 5 MRI Agent Bench and the offline comparison with and without MRFoundry. A: public Core examples require inversion-recovery sequence simulation, reconstruction and T1 fitting (SEQ-01), or PDFF/R2* reconstruction from multi-coil, multi-echo k-space (RECON-01) [10]. Icons are illustrative; these tasks differ from Figures 3-4. B: paired attempts use the same model and task instance in separate sandboxes, with matched inputs, tools and time limit. The intervention is seven frozen skills plus their discovery instruction; skill reading is agent-selected. Web search is disabled, with package-index access allowed. Each arm produces artifacts assessed by the same checks; empty checklist symbols do not denote success. C-D: saved all-checks-passed rates on common 0-100% scales. Change is with minus without skills, in percentage points; boxes report the saved two-way task/model cluster-bootstrap 95% intervals. Core covers 24 task types × ten models (240 pairs). Hard combines three models × 15 types × two instances (90 pairs) and four models × 11 multi-stage types × one instance (44 pairs). Its interval includes zero, and unequal coverage precludes a common-task model ranking. Supporting Information gives resampling details, secondary unclustered tests and domain/cohort breakdowns. No model or grader was rerun.

selected panel. Supporting Information gives the original algorithm, secondary unclustered McNemar comparisons, task validation, grading, and budgets.

All model-reaching trials for the ten analyzed models were retained. Nine started pairs for an eleventh model were excluded after its API rejected the host's editing tool. We restarted 143 pair setups (94 Core, 49 Hard) that failed before either model turn, keeping task/model conditions unchanged. Memory and disk policies changed during evaluation. Documented memory limits were matched within newly prepared pairs, but records do not establish uniform resources throughout the study. Supporting Information documents policy changes, missing exposure records, and a retained Hard image-subcheck weakness.

3 | RESULTS

3.1 | Research demonstrations

3.1.1 | Sequence and simulation checks

Figure 3 follows the research request through sequence construction, simulation, and inspection. Readback found no timing errors; maximum per-axis gradient and

TABLE 2 MRI Agent Bench comparison by model and evaluation cohort

	Core: 24/model		Hard: 30/model		Hard: 11/model	
Model	**No skills**	**With MRFoundry**	**No skills**	**With MRFoundry**	**No skills**	**With MRFoundry**
Claude Haiku 5.5	15	19	-	-	2	3
Claude Opus 5.5	22	22	16	20	-	-
Claude Sonnet 5.5	17	23	-	-	-	-
Gemini 3.8 Flash	19	22	-	-	2	4
Gemini 4 Argon	21	24	13	14	-	-
GPT-5.3 Codex	9	13	-	-	1	1
GPT-6.1 Sol	19	22	15	18	-	-
GPT-6.1 Terra	22	22	-	-	-	-
Kimi K3	13	16	-	-	-	-
MAI Code 1.1 Flash	5	5	-	-	0	0
Completed	**162**	**188**	**44**	**52**	**5**	**8**
Paired trials	240		90		44	
Discordant pairs	39 / 13		16 / 8		4 / 1	

MRFoundry: frozen evaluated snapshot (Supporting Information). Entries are verified completions; every automated check must pass. Models are ordered alphabetically. A dash denotes an unevaluated cohort. Core: one instance of each of 24 task types per model. Hard 30/model: two instances of all 15 types; Hard 11/model: one instance of 11 multi-stage types. Discordant pairs list completions only with MRFoundry / only with no skills. Hard coverage differs, so the two cohorts do not form a common model ranking. Pooled Hard: 49 to 60 completions among 134 pairs; system-design subset: 1 to 3 among 24. Aggregate effect intervals are shown in Figure 5.

slew were 12.735 mT/m and 125.266 T/m/s, within configured limits. The 128 spokes contained 8,192 ADC samples. Exported/prewrite coordinates differed by at most 0.00162 cycles/m, and halving the RF step changed the complex signal by 0.07096%.

The resulting Pulseq file, trajectory, and simulation records made these checks inspectable at each connection between tools. They support timing, trajectory transfer, and one temporal-discretization choice. They do not establish full physical-model convergence, scanner readiness, or diagnostic image quality. The researcher's intervention in simulator selection remains part of the demonstrated workflow.

3.1.2 | Sampling and reconstruction comparison

Source inspection found 195 acquired phase-encoding lines in the 256-column grid. Masks retained 110 and 67 acquired lines, giving effective accelerations of 1.77 and 2.91 (Figure 4). Outputs included complex reconstructions, masks, sensitivity maps, and source-linked summaries. All-source RSS matched the provider's reference with relative L2 error 8.85×10^{-8}, verifying source-image construction rather than undersampled reconstruction accuracy.

L1-wavelet reconstruction reduced magnitude discrepancy relative to the initial SENSE outputs, but zero-filled RSS remained closer to noisy all-source RSS at both settings. Substantial sampled- and withheld-line residuals persisted despite small changes with additional iterations. The saved comparison therefore documents an inspected revision with unresolved data agreement. Supporting Information retains complete diagnostics; the noisy reference does not establish image accuracy.

3.2 | A retained research lesson

The acquired-support record links the 195-line denominator to its source digest, release contract, and numerical report (Figure 2). Its lesson is to verify axes and acquired support before masking or calculating acceleration; 195 is source-specific. The initial calculation already used this denominator, so the record is not evidence of correcting an executed 256-based estimate. It demonstrates how a concrete decision can be retained with its scope and evidence for later revalidation, not improved performance on a subsequent task.

3.3 | MRI Agent Bench comparison

MRFoundry achieved 188 versus 162 completions across 240 Core pairs (78.3% versus 67.5%; difference 10.8 percentage points, 95% CI 0.8 to 21.2). Discordant pairs

comprised 39 with-only and 13 without-only completions (Figure 5). These counts show that the aggregate gain includes both improvements and regressions on individual pairs.

Hard completion rose from 49 to 60/134 (36.6% to 44.8%; difference 8.2 points, 95% CI −3.9 to 19.7), with 20 with-only and nine without-only completions. Its interval includes zero. Table 2 separates Hard cohorts because coverage differs. Supporting Information retains Core domain/model results, Hard cohorts, and token-use measures. The findings concern availability of the complete guidance package under the recorded execution conditions. Seven models improved in aggregate Core count and three were unchanged, although unchanged totals could contain different successful pairs. The model-level table makes that coverage explicit; the aggregate result is not a general model ranking.

4 | DISCUSSION

4.1 | Connecting MRI knowledge to agent actions

MRFoundry makes MRI expertise available in a form an existing agent can consult while working. Its practical contribution is the connection between a research question, relevant scientific sources, tool-specific actions, and output checks. Researchers can use a general skill to frame a task and combine specialist guidance as the work crosses domains. This supports incremental adoption within an existing software environment rather than requiring a new numerical stack. The plain-text organization also exposes instructions for inspection and revision: a researcher can follow a reference, examine a recommended check, or contribute a correction. This transparency is useful when a workflow depends on conventions that are easy to leave implicit, such as acquired support or the meaning of a reference image.

The Core comparison supports the usefulness of making this guidance available under the tested conditions. Hard evidence was less conclusive, and gains varied across tasks. Because the intervention bundled all seven skills and their discovery instruction, it cannot identify which components caused a change. The results motivate further study of guidance selection and use; they do not establish that more instruction always improves an agent.

4.2 | Evidence, supervision, and reusable lessons

The demonstrations show why intermediate checks matter. Sequence export requires timing and coordinate verification before simulated behavior can be interpreted. Reconstruction requires explicit sampling conventions and a reference whose limitations are understood. Established applications provide the numerical methods; guidance helps select their inputs, connect their outputs, and identify questions that remain unresolved.

Researcher involvement was substantive. The sequence example required intervention to adopt an established simulator, and reconstruction did not improve agreement over zero-filled RSS. These observations place the toolbox within supervised research practice. The value illustrated is an inspectable account of decisions and checks, rather than evidence of autonomous scientific discovery or consistently superior reconstruction.

Project memory extends this account across sessions. Self-Refine studies revision through textual feedback, and Reflexion retains feedback across attempts[46,47]. MRFoundry uses written project records to retain a lesson together with the evidence that limits its scope. Whether this improves later work requires longitudinal comparisons that measure retrieval, application, and resulting outcomes. Memory reuse and community contributions were not isolated here.

4.3 | Evaluation scope and future work

Tasks were agent-authored within an author-selected scope; Core shares problem structure with development tasks, and independent expert review is incomplete. One run per Core task-model combination does not establish repeated-run reliability. Hard coverage differs, and nominal reasoning settings do not equalize provider budgets. Automated checks measure specified outputs, not researcher effort, judgment, or clinical validity. Historical model snapshots, full execution configurations, and per-run resource exposure are unavailable. Setup incidents, shared resources, and the retained grading weakness further limit interpretation.

Future evaluation should add independent task review, repeated runs, and complete execution records, and test whether guidance transfers to new acquisition conditions and host environments. Longitudinal studies can assess whether recorded lessons improve subsequent decisions and whether reviewed contributions preserve that benefit across projects. The present demonstrations lack matched baselines and establish neither scanner safety nor diagnostic benefit. MRI Agent Bench provides

an accompanying resource for studying task completion, while broader research and clinical outcomes require their own evaluation designs.

5 | CONCLUSION

MRFoundry connects MRI knowledge, established software, and researcher supervision through reusable skills and evidence-linked project records. The demonstrations document a later skill-guided sequence audit and guidance use during reconstruction. Alongside the toolbox, MRI Agent Bench supports paired assessment of verified task completion. The Core comparison supports domain guidance under tested conditions; benefits from memory reuse and shared contributions remain unmeasured.

6 | DATA AND SOFTWARE AVAILABILITY

MRFoundry source and contribution guidance are available at https://github.com/KeWang0622/MRFoundry[7,34], with the evaluated snapshot preserved separately[45]. The reference index and tutorial provide entry points[31,48]. MRI Agent Bench publishes Core tasks, data, checkers, and result exports[10,11]. Hard materials, per-task outcomes, and traces remain private; only aggregates are reported. The paper evidence directory supplies original demonstration scripts, reports, input identities, recorded environments, and historical evaluation settings, with gaps and reproduction limits stated explicitly. Raw data remain in their original releases; M4Raw Brain v1.6 is CC BY 4.0[40].

7 | AI ASSISTANCE DISCLOSURE

AI tools supported software development, writing, and explanatory illustrations under the author's direction. Scientific MRI images and quantitative plots derive from recorded data and numerical outputs without generative enhancement. The author is responsible for the scientific content and final manuscript.

REFERENCES

1. Yao Shunyu, Zhao Jeffrey, Yu Dian, et al. ReAct: Synergizing Reasoning and Acting in Language Models. In: *International Conference on Learning Representations*; 2023.
2. Anthropic . How Claude Code works https://code.claude.com/docs/en/how-claude-code-works. Accessed November 3, 2026.
3. Yang John, Jimenez Carlos E., Wettig Alexander, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In: *Advances in Neural Information Processing Systems* 37:50528–50652; 2024.
4. Hong Sirui, Lin Yizhang, Liu Bang, et al. Data Interpreter: An LLM Agent for Data Science. In: *Findings of the Association for Computational Linguistics: ACL 2025*:19796–19821. Association for Computational Linguistics; 2025; Vienna, Austria.
5. Boiko Daniil A., MacKnight Robert, Kline Ben, Gomes Gabe. Autonomous chemical research with large language models. *Nature.* 2023;624(7992):570–578.
6. Bernstein Matt A., King Kevin F., Zhou Xiaohong Joe. *Handbook of MRI Pulse Sequences.* Academic Press; 2004.
7. Wang Ke. MRFoundry: MRI research guidance for coding agents https://github.com/KeWang0622/MRFoundry/tree/6e4408e61ebf073845c32d95eca083e21a5ed33a. Guidance snapshot 6e4408e; MIT license. Accessed November 3, 2026; 2026.
8. Zaiss Moritz, Aly Amr, Endres Jonathan, Dornstetter Tobias, Weinmüller Simon, Maier Andreas. Agentic MR sequence development: leveraging LLMs with MR skills for automatic physics-informed sequence development arXiv preprint arXiv:2604.13282. 2026.
9. Long Ziyang, Li Xinqi, Chen Junzhou, Gao Yifan, Li Debiao, Yang Hsin-Jung. BCER Agent: Reliable Long-Horizon MRI Workflow Execution via Compilation, Artifact Binding, and Bounded Local Recovery arXiv preprint arXiv:2605.29163. 2026.
10. Wang Ke. MRI Agent Bench public task directory https://github.com/KeWang0622/MRIAgentBench/tree/main/families. Accessed November 3, 2026; 2026.
11. Wang Ke. MRI Agent Bench Core v0.1.0 (released as BlochBench) https://github.com/KeWang0622/MRIAgentBench/releases/tag/core-v0.1.0. Historical release title: BlochBench Core v0.1.0 – research preview. Tag core-v0.1.0. Published November 3, 2026. Accessed November 3, 2026; 2026.
12. Anthropic . Extend Claude with skills https://code.claude.com/docs/en/skills. Accessed November 3, 2026.
13. OpenAI . Build skills https://learn.chatgpt.com/docs/build-skills. Accessed November 3, 2026.
14. Ravi Keerthi Sravan, Geethanath Sairam, Vaughan John Thomas, Jr. PyPulseq: A Python Package for MRI Pulse Sequence Design. *Journal of Open Source Software.* 2019;4(42):1725.
15. Castillo-Passi Carlos, Coronado Ronal, Varela-Mattatall Gabriel, Alberola-López Carlos, Botnar René, Irarrazaval Pablo. KomaMRI.jl: An open-source framework for general MRI simulations with GPU acceleration. *Magnetic Resonance in Medicine.* 2023;90(1):329–342.
16. Ong Frank, Lustig Michael. SigPy: A Python Package for High Performance Iterative Reconstruction. In: *Proceedings of the International Society for Magnetic Resonance in Medicine* 27:4819; 2019.
17. Garyfallidis Eleftherios, Brett Matthew, Amirbekian Bagrat, et al. Dipy, a library for the analysis of diffusion MRI data. *Frontiers in Neuroinformatics.* 2014;8:8.
18. Wang Ke. MRFoundry project-memory guide https://github.com/KeWang0622/MRFoundry/blob/6e4408e61ebf073

845c32d95eca083e21a5ed33a/skills/mri-research/references/project-memory.md. Accessed November 3, 2026; 2026.
19. Nichols Thomas E, Das Samir, Eickhoff Simon B, et al. Best practices in data analysis and sharing in neuroimaging using MRI. *Nature Neuroscience.* 2017;20(3):299-303.
20. Tejani Ali S., Klontzas Michail E., Gatti Anthony A., et al. Checklist for Artificial Intelligence in Medical Imaging (CLAIM): 2024 Update. *Radiology: Artificial Intelligence.* 2024;6(4):e240300.
21. Roemer P. B., Edelstein W. A., Hayes C. E., Souza S. P., Mueller O. M. The NMR phased array. *Magnetic Resonance in Medicine.* 1990;16(2):192–225.
22. Layton Kelvin J., Kroboth Stefan, Jia Feng, et al. Pulseq: A rapid and hardware-independent pulse sequence prototyping framework. *Magnetic Resonance in Medicine.* 2017;77(4):1544-1552.
23. Pruessmann Klaas P., Weiger Markus, Scheidegger Markus B., Boesiger Peter. SENSE: Sensitivity encoding for fast MRI. *Magnetic Resonance in Medicine.* 1999;42(5):952–962.
24. Uecker Martin, Lai Peng, Murphy Mark J., et al. ESPIRiT—an eigenvalue approach to autocalibrating parallel MRI: Where SENSE meets GRAPPA. *Magnetic Resonance in Medicine.* 2014;71(3):990-1001.
25. Lustig Michael, Donoho David, Pauly John M. Sparse MRI: The application of compressed sensing for rapid MR imaging. *Magnetic Resonance in Medicine.* 2007;58(6):1182–1195.
26. Uecker Martin, Ong Frank, Tamir Jonathan I., et al. Berkeley Advanced Reconstruction Toolbox. In: *Proceedings of the International Society for Magnetic Resonance in Medicine* 23:2486; 2015.
27. Knoll Florian, Zbontar Jure, Sriram Anuroop, et al. fastMRI: A Publicly Available Raw k-Space and DICOM Dataset of Knee Images for Accelerated MR Image Reconstruction Using Machine Learning. *Radiology: Artificial Intelligence.* 2020;2(1):e190007.
28. Sriram Anuroop, Zbontar Jure, Murrell Tullie, et al. End-to-End Variational Networks for Accelerated MRI Reconstruction. In: *Medical Image Computing and Computer Assisted Intervention – MICCAI 2020*:64–73; 2020.
29. Basser Peter J., Mattiello James, LeBihan Denis. MR diffusion tensor spectroscopy and imaging. *Biophysical Journal.* 1994;66(1):259–267.
30. Chung S. W., Lu Ying, Henry Roland G. Comparison of bootstrap approaches for estimation of uncertainties of DTI parameters. *NeuroImage.* 2006;33(2):531–541.
31. Wang Ke. MRFoundry reference index https://github.com/KeWang0622/MRFoundry/blob/6e4408e61ebf073845c32d95eca083e21a5ed33a/REFERENCES.md. Accessed November 3, 2026; 2026.
32. Vercel . Skills: the open agent skills tool https://github.com/vercel-labs/skills. Accessed November 3, 2026.
33. Wang Ke. MRFoundry application setup guide https://github.com/KeWang0622/MRFoundry/blob/6e4408e61ebf073845c32d95eca083e21a5ed33a/skills/mri-research/references/tool-setup.md. Accessed November 3, 2026; 2026.
34. Wang Ke. Contributing to MRFoundry https://github.com/KeWang0622/MRFoundry/blob/6e4408e61ebf073845c32d95eca083e21a5ed33a/CONTRIBUTING.md. Accessed November 3, 2026; 2026.
35. Harris Charles R., Millman K. Jarrod, van der Walt Stéfan J., et al. Array programming with NumPy. *Nature.* 2020;585(7825):357–362.
36. Virtanen Pauli, Gommers Ralf, Oliphant Travis E., et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods.* 2020;17(3):261–272.
37. Hunter John D. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering.* 2007;9(3):90–95.
38. Winkelmann Stefanie, Schaeffter Tobias, Koehler Thomas, Eggers Holger, Doessel Olaf. An optimal radial profile order based on the Golden Ratio for time-resolved MRI. *IEEE Transactions on Medical Imaging.* 2007;26(1):68–76.
39. Lyu Mengye, Mei Lifeng, Huang Shoujin, et al. M4Raw: A multi-contrast, multi-repetition, multi-channel MRI k-space dataset for low-field MRI research. *Scientific Data.* 2023;10:264.
40. Lyu Mengye, Mei Lifeng, Huang Shoujin, et al. M4Raw: A multi-contrast, multi-repetition, multi-channel MRI k-space dataset for low-field MRI research [V1.6] Zenodo. CC BY 4.0; accessed November 3, 2026; 2023.
41. h5py contributors . h5py 3.16.0 documentation https://docs.h5py.org/. Accessed November 3, 2026.
42. Lyu Mengye, M4Raw contributors . M4Raw Brain H5 contract https://huggingface.co/datasets/mylyu/M4Raw_brain/blob/v1.6.0/H5_FORMAT.md. Release v1.6.0 format documentation. Accessed November 3, 2026; 2026.
43. SigPy contributors . SigPy L1WaveletRecon implementation https://github.com/mikgroup/sigpy/blob/v0.1.27/sigpy/mri/app.py. Version 0.1.27; db4 default and inverse objective. Accessed November 3, 2026.
44. Lee Gregory R., Gommers Ralf, Waselewski Filip, Wohlfahrt Kai, O'Leary Aaron. PyWavelets: A Python package for wavelet analysis. *Journal of Open Source Software.* 2019;4(36):1237.
45. Wang Ke. MRFoundry: evaluated code snapshot https://github.com/KeWang0622/MRFoundry/tree/bd49b9556146bcd70ab1026c52d67713f4cf2e7f. Frozen evaluated source snapshot bd49b9556146bcd70ab1026c52d67713f4cf2e7f, 31 October 2026; 2026.
46. Madaan Aman, Tandon Niket, Gupta Prakhar, et al. Self-Refine: Iterative Refinement with Self-Feedback. In: Oh A., Naumann T., Globerson A., Saenko K., Hardt M., Levine S., eds. *Advances in Neural Information Processing Systems*, 36:46534–46594. Curran Associates, Inc.; 2023.
47. Shinn Noah, Cassano Federico, Gopinath Ashwin, Narasimhan Karthik, Yao Shunyu. Reflexion: language agents with verbal reinforcement learning. In: Oh A., Naumann T., Globerson A., Saenko K., Hardt M., Levine S., eds. *Advances in Neural Information Processing Systems*, 36:8634–8652. Curran Associates, Inc.; 2023.
48. Wang Ke. MRFoundry interactive tutorial https://kewang0622.github.io/slides/mrfoundry/. Accessed November 3, 2026.